Ain Shams Engineering Journal 17 (2026) 104063

Contents lists available at ScienceDirect

Ain Shams Engineering Journal

journal homepage: www.sciencedirect.com

Full length article

Constrained optimization in physics-informed neural networks for singular three-point boundary value problems

Uzma Nadeem [a], Malik Ahsin Iqbal [a], Sadique Rehman [b], Aamir Farooq [c,d,e,*], Hijaz Ahmad [f,g,h,i], Fuad A. Awwad [j], Emad A.A. Ismail [j]

[a] Abdus Salam School of Mathematical Sciences, GC University, Lahore 54600, Pakistan
[b] Division of Mathematical and Physical Sciences, Kanazawa University, Kakuma, Kanazawa 920-1192, Japan
[c] Department of Mathematics, Zhejiang Normal University, Jinhua, 321004, PR China
[d] Research Center of Astrophysics and Cosmology, Khazar University, Baku, 41 Mehseti Street, AZ1096, Azerbaijan
[e] Department of Applied Mathematics and Data Science, Asian University for Women, Chattogram 4000, Bangladesh
[f] Irfan Suat Gunsel Operational Research Institute, Near East University, Nicosia/TRNC, 99138 Mersin 10, Turkey
[g] Sustainability Competence Centre, Széchenyi István University, Egyetem tér 1, H-9026 Győr, Hungary
[h] VIZJA University, Okopowa 59, 01-043 Warsaw, Poland
[i] Department of Mathematics, College of Science, Korea University, 145 Anam-ro, Seongbuk-gu, Seoul 02841, South Korea
[j] Department of Quantitative Analysis, College of Business Administration, King Saud University, P.O. Box 71115, Riyadh 11587 Saudi Arabia

ARTICLE INFO

Keywords:
Singular three-point boundary value problems
Physics-informed neural networks
Constrained optimization
Regularization techniques
Numerical approximation
Statistical evaluation

ABSTRACT

Physics-informed neural networks represent a category of deep learning models that directly incorporate physical laws into the training process to solve differential equations, thereby diminishing the dependence on extensively labeled datasets. This study investigates a constrained optimization framework within PINNs to address singular three-point boundary value problems, which present significant challenges owing to singularities and internal boundary conditions that result in non-standard solution behavior. To address these complexities, we developed a customized Physics-informed neural network architecture that integrates constraint-driven regularization terms into the loss function to enhance the generalization and numerical stability. The proposed approach was evaluated across multiple benchmark problems, with performance assessed using statistical metrics and the mean squared error. The optimization and training PINN regular framework will stabilize the training and convergence in the presence of singularities to yield dependable TPS-BVP solutions. The predicted solutions were rigorously compared with exact analytical solutions. The results demonstrate that the constrained optimization-based Physics-informed neural networks framework provides highly accurate and stable approximations, validating its effectiveness in handling complex singular boundary value problems.

1. Introduction

Multipoint singular boundary value problems (MPS-BVPs) address challenges within physics, engineering, applied mathematics, and other scientific disciplines. Numerous scholars have investigated two-point singular boundary value problems [4]. These BVPs are instrumental in modeling phenomena related to diffusion reactions [21], chemical kinetics, heat-transfer processes [29], thermal expansion, and fluid mechanics. Solutions to two-point singular BVPs can be derived through various methodologies, including the Reproducing Kernel Hilbert Space (RKHS) method [3] and the optimal homotopy analysis method [44]. Exploration of MPS-BVPs was pioneered by Moiseev [28]. The existing literature has a paucity of techniques for resolving singular three-point and multipoint problems. Three-point singularity boundary-value problems (TPS-BVPs) [36] represent an extension of BVPs, wherein boundary conditions are specified at three distinct points. These conditions may involve both endpoints and an interior point or all three may be interior points. Additionally, the domain may contain one or more singular points. Gupta [24] examined the stability of TPS-BVPs for second-order ordinary differential equations. Geng employed the Hilbert space scheme to solve such problems [20]. The general form of the TPS-BVPs

* Corresponding author at: Department of Mathematics, Zhejiang Normal University, Jinhua, 321004, PR China.
Email addresses: uzma.nadeem_22@sms.edu.pk (U. Nadeem), ahsin.iqbal_22@sms.edu.pk (M.A. Iqbal), sadique@stu.kanazawa-u.ac.jp (S. Rehman), aamirf@zjnu.edu.cn (A. Farooq), hijazahmad@korea.ac.kr (H. Ahmad), fawwad@ksu.edu.sa (F.A. Awwad), emadali@ksu.edu.sa (E.A.A. Ismail).

https://doi.org/10.1016/j.asej.2026.104063
Received 2 June 2025; Received in revised form 26 January 2026; Accepted 13 February 2026
Available online 9 March 2026
2090-4479/© 2026 The Authors. Published by Elsevier B.V. on behalf of Faculty of Engineering, Ain Shams University. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

[39] is expressed as follows:

$$p(t)\frac{d^2y(t)}{dt^2} + q(t)\frac{dy(t)}{dt} + r(t)y(t) = g(t, y(t)), \quad 0 \le t \le 1,$$
$$y(0) = 1, \quad y(1) = \epsilon y(\theta) + \beta, \tag{1}$$

where $p(t) \in C^2[0,1], q(t) \in C^1[0,1], r(t) \in C[0,1], [0,1], p(0) = p(1) = 0, q(0) = q(1) \neq 0, p(t) \neq 0, t \in (0,1), r(0) = r(1) \neq 0, 0 < \theta < 1$ and $q(1) - p'(0) \neq 0$. In (1), the singularity appears at different values of $t = 0, 1$. TPS-BVPs are common in real-world applications involving diffusion, heat transfer, and reaction-diffusion processes, especially when certain conditions are applied at different locations across a physical domain. Such problems include the study of heat conduction through extended surfaces, nonlinear chemical reaction models, electrohydrodynamic stability of charged fluids, biological population dynamics, and fluid flow through porous or stratified media. Due to boundary conditions or variable physical properties, singularities might occur in certain regions of the domain. The proposed PINN-based approach is novel for singularity and nonlocal boundary problems because it does not use mesh refinements or symbolic derivatives to discretize the domain.

In recent years, substantial attention has been directed toward developing stochastic-based computational approaches for solving singular and non-singular boundary value problems (BVPs) arising from various mathematical models [10,43,48]. Recently, a hybrid scheme based on feed-forward Gudermannian Neural Networks (GNN), combined with the global search efficiency of Genetic Algorithms (GA) [2] and the rapid local convergence capability of the Active-Set Method (ASM), has been explored for addressing the second kind of three-point singular boundary value problems (TPS-BVPs) [45]. However, the primary limitation of this approach lies in its high computational cost, as repeated derivations of the GNN activation function are required to compute the objective function multiple times, particularly for statistical analysis, making the process time-consuming and expensive.

Traditional numerical techniques such as finite difference, finite element, and spectral methods require extensive mesh generation, symbolic differentiation, and stepwise discretization, which can make them inefficient or unstable when dealing with nonlinear or singular boundary value problems. Neural networks [41], particularly Physics-Informed Neural Networks (PINNs), offer a mesh-free, data-efficient, and differentiable alternative with built-in enforcement of governing physics. By leveraging automatic differentiation, PINNs can compute higher-order derivatives without discretization errors and can approximate complex nonlinear functions, enabling accurate prediction of system behavior even in the presence of singularities or nonlocal boundary conditions. These capabilities make NN-based solvers robust and flexible alternatives to traditional numerical approaches, especially for complex boundary value problems. Building upon this motivation, PINNs [40,41] have emerged as a powerful paradigm that unifies machine learning and scientific computing by embedding governing physical laws directly into neural network training. Unlike conventional data-driven networks, PINNs incorporate differential equations, boundary, and initial conditions into the loss function, ensuring physically consistent and interpretable solutions [17,25]. This framework enables automatic differentiation, efficient handling of complex geometries, and accurate approximation of nonlinear systems without explicit discretization or symbolic derivation, establishing PINNs as a robust alternative to conventional solvers. Over the past few years, PINNs have rapidly evolved into a versatile computational tool for solving a wide range of differential equations. However, recent studies have emphasized the necessity for enhanced optimization and constraint-handling strategies to improve convergence and stability in challenging boundary-value problems. For instance, Son et al. [46] proposed the Augmented Lagrangian PINN (AL-PINN) framework, incorporating equality and inequality constraints directly into the loss function to stabilize training and strengthen boundary enforcement. Similarly, Bischof and Kraus [7] introduced a multi-objective loss-balancing strategy in computer methods in applied mechanics and engineering, demonstrating improved accuracy for multi-physics systems. Lu et al. [32] presented NSGA-PINN, which employs the Non-Dominated Sorting Genetic Algorithm II (NSGA-II) to perform multi-objective optimization during training, effectively balancing data-fit and physical consistency. Further advancements by Zhang et al.[51] combined the Kolmogorov-Arnold Network (KAN) architecture with augmented Lagrangian functions to accelerate convergence and enhance the representation of highly nonlinear PDEs. Finally, Guo et al. [23] provided a comprehensive review of recent progress in PINNs, highlighting developments in adaptive sampling, hybrid architectures, and optimization-driven formulations. Collectively, these contributions underscore the growing emphasis on constrained and optimization-enhanced formulations in PINNs. For broader perspectives on physics-informed learning and its applications, see the references cited in these works, which provide valuable insights into emerging methodologies and multi-disciplinary extensions of PINNs.

To further improve the generalization, stability, and robustness of the PINN framework, this study incorporates regularization techniques as part of a constrained optimization [27] strategy. Specifically, the loss function is augmented with penalty terms that constrain the network weights, helping to mitigate overfitting and improve convergence in the presence of singularities. The overall loss function of the PINN is structured as a weighted sum of the residual and boundary losses, where the residual term enforces the governing differential equations and the boundary term satisfies the three-point singular conditions. Automatic differentiation allows the model to compute all requisite derivatives in the residual term and thus obtain physically plausible solutions, which is a hallmark of learned rational outcomes. This integrated loss model [31] focuses the PINN on gauging the true solution space, as well as preserving differentiability and regularity in the singular regions.

The proposed model is trained using the Adam optimization algorithm [47] and is validated on three benchmark TPS-BVPs. Model accuracy is assessed by comparing the predicted solutions with exact analytical results, using different statistical metrics as the primary performance metric. The customized and regularized loss function is designed to effectively handle the internal singularities and three-point boundary structure, significantly enhancing solution accuracy and model reliability [12,13]. Furthermore, this study investigates the impact of different network configurations, including variations in the number of hidden layers (20, 30, and 50), to evaluate trade-offs between computational efficiency and predictive accuracy [11]. The hyperbolic tangent (tanh) activation function is primarily used due to its ability to capture smooth and singular features in boundary solutions [1]. Additional activation functions, including sigmoid, SeLU, and ReLU, are explored to assess the robustness and adaptability of the PINN framework under regularized training conditions [5,42]. In contrast to traditional PINNs, this framework implements regularization and tensioned optimization directly into the loss function, which provides solution smoothness and mitigates singular region overfitting.

The structure of the paper is organized as follows: Section 2 describes the methodology and implementation details of the proposed PINN framework. Section 3 presents the numerical experiments to evaluate the model's performance. Section 4 discusses the results in detail and provides comprehensive insights. Finally, Section 5 outlines the concluding remarks and suggests potential directions for future research.

2. Methodology for solving TPS-BVPs

This section presents an efficient computational framework based on PINNs to approximate the solution of TPS-BVPs. The central concept of PINNs lies in embedding the governing physical laws into the loss function of a Deep Neural Network (DNN), enabling the model to satisfy the differential equation and the boundary conditions simultaneously. Consider the general second-order TPS-BVP as:

$$\zeta\left(t, y(t), y'(t), y''(t)\right) = 0, \quad t \in \overline{D} \subset \mathbb{R}, \tag{2}$$

where $y(t)$ denotes the unknown solution, $t \in [0,1]$ is the independent variable, and $\overline{D}$ represents the discretized domain comprising a finite set of collocation points. The corresponding three-point boundary conditions are given by:

$$y(0) = \alpha, \quad y(\xi) = \beta, \quad y(1) = \gamma, \tag{3}$$

where $0 < \xi < 1$ is an interior point, and α, β, γ are specified constants. To approximate the solution $y(t)$, a feed-forward DNN consists of an input layer, multiple hidden layers with k neurons each, and a linear output layer. The neural network output $\hat{y}_{\mathrm{NN}}(t;\Theta)$ is mathematically expressed as:

$$\hat{y}_{\mathrm{NN}}(t;\Theta) = (z_J \circ z_{J-1} \circ \ldots \circ z_1)(t), \tag{4}$$

where $\Theta = \{W^{(j)}, b^{(j)}\}_{j=1}^{J}$ denotes all trainable weights and biases, and

$$z_j(t^{j-1}) = W^{(j)} t^{j-1} + b^{(j)}, \quad j = 1, 2, \ldots, J. \tag{5}$$

This work utilizes the hyperbolic tangent (tanh) activation function in all hidden layers due to its better performance and stable gradient properties during training. The model parameters Θ are optimized by minimizing the total loss function based on the mean squared error (MSE), which integrates contributions from the equation residual, boundary conditions, and available data. The total loss is defined as:

$$\mathcal{L}(\Theta) = \mathcal{L}_f + \mathcal{L}_{bc} + \mathcal{L}_d, \tag{6}$$

where:$\mathcal{L}_f$: Residual loss at N_r interior points:

$$\mathcal{L}_f = \frac{1}{N_r} \sum_{j=1}^{N_r} \left| \zeta(t_j;\Theta) \right|^2, \tag{7}$$

$\mathcal{L}_{bc}$: Boundary condition loss:

$$\mathcal{L}_{bc} = \frac{1}{3} \left(\left| \hat{y}_{\mathrm{NN}}(0;\Theta) - \alpha \right|^2 + \left| \hat{y}_{\mathrm{NN}}(\xi;\Theta) - \beta \right|^2 + \left| \hat{y}_{\mathrm{NN}}(1;\Theta) - \gamma \right|^2 \right), \tag{8}$$

$\mathcal{L}_d$: Data loss (if applicable):

$$\mathcal{L}_d = \frac{1}{N_d} \sum_{j=1}^{N_d} \left| \hat{y}_{\mathrm{NN}}(t_j;\Theta) - y_{\mathrm{data}}(t_j) \right|^2. \tag{9}$$

The optimization uses the Adam algorithm [8], which updates Θ iteratively until convergence is achieved. This PINN framework efficiently handles the singularities and multiple boundary conditions present in TPS-BVPs. It eliminates the need for repeated symbolic differentiation and allows flexibility in solving nonlinear and complex problems. Adam was the main focus, yet L-BFGS and RMSProp were used on a few selected examples. L-BFGS was able to speed up convergence in the smooth areas of the function, but Adam was more stable in the regions with singularities. RMSProp behaved similarly to Adam, but more careful tuning of the learning rate was needed.

2.1. Adaptive physics-informed neural networks (A-PINNs)

Although the classical PINN framework efficiently embeds the governing physics into the loss function, its performance may deteriorate when the training objective becomes highly imbalanced. In TPS-BVPs, the residual loss and boundary losses may differ by several orders of magnitude during training, causing the optimizer to focus disproportionately on one term and leading to slow convergence or inaccurate boundary satisfaction. To overcome this issue, we implemented an adaptive physics-informed neural network (A-PINN) formulation in which the contributions of different loss components are automatically re-weighted during training. This adaptive strategy follows the self-adaptive weighting concept introduced in [34] and is consistent with the gradient-balancing principles discussed in [50].

Instead of using the fixed loss in Eq. (6), the adaptive loss takes the form

$$\mathcal{L}_{\mathrm{A\text{–}PINN}}(\Theta) = \omega_f \, \mathcal{L}_f + \omega_{bc} \, \mathcal{L}_{bc} + \omega_d \, \mathcal{L}_d, \tag{10}$$

where ω_f, ω_{bc} and ω_d are non-negative trainable coefficients that balance the relative importance of the interior physics residual, boundary constraints, and data fitting terms, respectively. The adaptive nature of A-PINNs lies in the fact that these weights are updated simultaneously with the network parameters, such that each loss component contributes comparably during optimization.

Following the gradient-norm balancing strategy, the weights are updated to minimize the difference between the magnitudes of the parameter gradients generated by each loss term. Denoting by $\nabla_\Theta \mathcal{L}_i$ the gradient of the i-th loss component ($i \in \{f, bc, d\}$), the adaptive weights satisfy

$$\omega_i \propto \frac{1}{\left\| \nabla_\Theta \mathcal{L}_i \right\| + \varepsilon}, \tag{11}$$

where ε is a small positive constant that prevents division by zero. In this way, large-magnitude gradient terms receive smaller weights, while under-represented terms are amplified. The weights were normalized after each update step to preserve numerical stability.

The A-PINN formulation preserves the architecture, activation function, and optimization strategy of the standard PINN framework while dynamically enforcing a balanced contribution from all physical constraints. This adaptive weighting significantly improves convergence and prevents the residual loss from dominating the learning process, particularly in the presence of interior singularities or steep gradients, as is commonly encountered in TPS-BVPs.

2.2. Regularization techniques

TPS-BVPs often involve sharp gradients or singularities within the domain, which pose challenges for standard neural network approximations. These singularities can lead to divergence or instability during the training process. To mitigate such issues, regularization techniques are employed to enhance the generalization and numerical stability of the model. Regularization introduces penalty terms into the loss function, which comprises both the residuals of the differential equations and the boundary conditions. These penalties help prevent overfitting, particularly near critical points where accurate approximation is essential.

In this work, we implement two commonly used regularization strategies, namely L_1 and L_2 regularization, to evaluate their effect on the performance of the proposed PINN framework. The L_1 regularization promotes sparsity in the weight matrices by driving some of the weights toward zero, thereby reducing model complexity and improving generalization capability. This leads to a more flexible and robust approximation with minimal error. Mathematically, the L_1 regularization term is expressed as:

$$L_1 = \lambda \sum_{i=1}^{N} |W_i|, \tag{12}$$

where λ is the regularization parameter. This regularization parameter was set through empirical tuning; a value of $\lambda = 10^{-3}$ offered the best balance between convergence stability and accuracy in preliminary tests. Flexibility was preserved by lower values while larger values strengthen smoothness. For consistency, the number of collocation points was set at 1500 for all examples. In L_1 regularization the

Table 1
Performance metrics used to evaluate model predictions against observations.

Metric	Definition	Range/Notes
MSE	$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$	≥ 0 (lower is better)
RMSE	$\mathrm{RMSE} = \sqrt{\mathrm{MSE}}$	Same units as y; ≥ 0
MAE	$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}\lvert y_i - \hat{y}_i\rvert$	≥ 0 (robust to outliers vs. MSE)
MAPE	$\mathrm{MAPE} = \frac{100}{n}\sum_{i=1}^{n}\left\lvert\frac{y_i - \hat{y}_i}{y_i}\right\rvert$	% error; undefined if any $y_i = 0$
R^2	$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$	$(-\infty, 1]$; 1 is perfect
R (Pearson)	$R = \frac{\sum_{i=1}^{n}(y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}\sqrt{\sum_{i=1}^{n}(\hat{y}_i - \bar{\hat{y}})^2}}$	$[-1, 1]$; linear association
RE (ℓ_2)	$\mathrm{RE} = \frac{\lVert y - \hat{y}\rVert_2}{\lVert y\rVert_2}$	≥ 0; scale-free
rSD	$\mathrm{rSD} = \frac{\mathrm{Std}(y - \hat{y})}{\bar{y}} \times 100$	%; depends on $\bar{y} \neq 0$
NSE	$\mathrm{NSE} = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$	$(-\infty, 1]$; 1 is perfect

Notation: n is the number of samples; y_i and $\hat{y}_i$ denote observed and predicted values; $\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$ and $\bar{\hat{y}} = \frac{1}{n}\sum_{i=1}^{n}\hat{y}_i$; $\lVert\cdot\rVert_2$ is the Euclidean norm; $\mathrm{Std}(\cdot)$ is the sample standard deviation.

penalty term added to the loss function is just the absolute value of all the weights. Formulating the L_1 technique with the cost function, is given as

$$\mathcal{L}_f = \frac{1}{N_r}\sum_{j=1}^{N_r}\zeta(t_j;\Theta) + \frac{1}{3}\left(\left|\hat{y}_{\mathrm{NN}}(0;\Theta) - \alpha\right|^2 + \left|\hat{y}_{\mathrm{NN}}(\xi;\Theta) - \beta\right|^2 + \left|\hat{y}_{\mathrm{NN}}(1;\Theta) - \gamma\right|^2\right) + \lambda\sum_{i=1}^{N}|W_i| \tag{13}$$

Talking about the L_2 regularization, also known as ridge regularization, it works by penalizing the large values of weights by using the L_2 squared norm.

$$L_2 = \lambda\sum_{i=1}^{N} W_i^2. \tag{14}$$

This L_2 squared norm acts as a penalty term being added to the loss function of the TPS-BVPs, given by:

$$\mathcal{L}_f = \frac{1}{N_r}\sum_{j=1}^{N_r}\zeta(t_j;\Theta) + \frac{1}{3}\left(\left|\hat{y}_{\mathrm{NN}}(0;\Theta) - \alpha\right|^2 + \left|\hat{y}_{\mathrm{NN}}(\xi;\Theta) - \beta\right|^2 + \left|\hat{y}_{\mathrm{NN}}(1;\Theta) - \gamma\right|^2\right) + \lambda\sum_{i=1}^{N} W_i^2. \tag{15}$$

L_2 weight decay and early stopping were implemented converging more slowly and stabilizing overfitting. These stabilizing techniques ensured that the convergence was smooth with no extreme fluctuations around the mean of the learned parameters. With overfitting in physics-informed neural networks, other regularization techniques were used, like gradient-norm penalties, Sobolev type losses and adaptive loss weighting. These techniques were analyzed conceptually and are promising next steps in this line of work.

2.3. Statistical measures

This section provides a detailed study of the statistical analysis [22] used to measure the performance of the model for solving TPS-BVPs with the help of statistical techniques. We implemented statistical matrices such as Mean Square Error (MSE), Root Mean Square Error (RMSE) [26], Mean Absolute Error (MAE) [16], Mean Absolute Percentage Error (MAPE) [14], R^2, which is the determination coefficient [15], similarly R a Pearson's correlation coefficient [6], relative standard deviation (rSD) [38], Nash-Sutcliffe Efficiency (NSE) [35] and Relative Error (RE) [37]. These measures aim to evaluate the model's performance, which quantifies the accuracy of the neural network-based solver. The following are the mathematical formulations of each statistical metric used in this work, see Table 1:

To assess the comprehensive evaluation of model performance, the metrics that focused on measures of dispersion are MSE, RMSE and MAE which evaluate the overall accuracy of the model, while the metrics that assess the differences of proportions are MAPE and RE, focuse on measuring the relative difference. The measures of association i.e., R^2, NSE and rSD reflected consistency and predictive stability.

3. Numerical experiments

This section presents the numerical experiments carried out to validate the effectiveness and accuracy of the proposed PINN framework in solving different examples of TPS-BVPs. The proposed network was implemented using the TensorFlow library for its construction and automatic differentiation for optimization [18]. The NumPy library was utilized for numerical computation and evaluation of the exact solution. After all the calculations, Matplotlib was used to plot the graphs of comparisons and mean square loss. In each case, the PINN model is trained to minimize the total loss function, incorporating the equation residual loss [19] and the boundary condition loss [30]. The optimal set of network parameters is determined through the training process, enabling the model to capture the underlying solution behavior effectively. The following examples of TPS-BVPs are from the recent work presented in [45]. Each example and its corresponding boundary conditions are provided, and the associated loss functions within the PINNs framework are formulated. All the considered problems are second-order boundary value problems (BVPs) [33] involving trigonometric and hyperbolic trigonometric functions such as sin, cos, sinh, and cosh. The presence of these forcing terms reflects the oscillatory nature [49] of the solution and highlights the possible boundary layer effects within the physical system [9].

Example 3.1. Consider the second type of TPS-BVP, which involves a trigonometric function and is expressed as:

$$\frac{d^2y(t)}{dt^2} + \frac{2}{t}\frac{dy(t)}{dt} + \frac{\pi^2 t\sin\pi t}{t} - \frac{2\pi\cos\pi t}{t} = 0, \quad y(0) = 0, \quad y(1) = -0.5 + 0.5y(0.5). \tag{16}$$

with boundary conditions while the exact solution is given in the form of a trigonometric function $y(t) = \sin\pi t$. It is noted from Eq. (16) that the singularity arises only at $t = 0$. Based on the definition of the total loss function in Eq. (6), the corresponding loss function for Eq. (16), along with its associated boundary conditions, is formulated as:

$$\mathcal{L}_f = \left|\frac{d^2y(t_i)}{dt_i^2} + \frac{2}{t_i}\frac{dy(t_i)}{dt_i} + \frac{\pi^2 t_i\sin\pi t_i}{t_i} - \frac{2\pi\cos\pi t_i}{t_i}\right|^2,$$

$$\mathcal{L}_{\mathrm{bc}} = |y(0)|^2 + |y(1) + 0.5 - 0.5y(0.5)|^2,$$

$$\mathcal{L} = \left|\frac{d^2y(t_i)}{dt_i^2} + \frac{2}{t_i}\frac{dy(t_i)}{dt_i} + \frac{\pi^2 t_i\sin\pi t_i}{t_i} - \frac{2\pi\cos\pi t_i}{t_i}\right|^2 + |y(0)|^2 + |y(1) + 0.5 - 0.5y(0.5)|^2.$$

Example 3.2. Consider the second type of TPS-BVP, characterized by multiple singularities and involving hyperbolic trigonometric functions, which is formulated as:

$$\frac{d^2y(t)}{dt^2} + \frac{1}{t}\frac{dy(t)}{dt} + \frac{y(t)}{t(1-t)} - \frac{\sinh t}{t(1-t)} - \frac{\cosh t}{t} - \sinh t = 0, \tag{17}$$

$$y(0) = 0, \quad y(1) + 0.5y(0.8) = \sinh 1 + 0.5\sinh\left(\frac{4}{5}\right). \tag{18}$$

with the boundary conditions, while the exact solution is $y(t) = \sinh t$. Eq. (18) has a singularity at two points, $t = 0$ and $t = 1$. Based on the definition of the total loss function in Eq. (6), the corresponding loss function for Eq. (18), along with its associated boundary conditions, is formulated as:

$$\mathcal{L}_f = \left| \frac{d^2y(t_i)}{dt_i^2} + \frac{1}{t_i}\frac{dy(t_i)}{dt_i} + \frac{y(t_i)}{t_i(1-t_i)} - \frac{\sinh t_i}{t(1-t_i)} - \frac{\cosh t_i}{t_i} - \sinh t_i \right|^2,$$

$$\mathcal{L}_{bc} = |y(0)|^2 + \left| y(1) - \sinh(1) + 0.5y(0.8) - 0.5\sinh\left(\frac{4}{5}\right) \right|^2,$$

$$\mathcal{L} = \left| \frac{d^2y(t_i)}{dt_i^2} + \frac{1}{t_i}\frac{dy(t_i)}{dt_i} + \frac{y(t_i)}{t_i(1-t_i)} - \frac{\sinh t_i}{t_i(1-t_i)} - \frac{\cosh t_i}{t_i} - \sinh t_i \right|^2 + |y(0)|^2 + \left| y(1) - \sinh(1) + 0.5y(0.8) - 0.5\sinh\left(\frac{4}{5}\right) \right|^2.$$

Example 3.3. Consider the nonlinear second type of TPS-BVP, characterized by multiple singularities and involving hyperbolic trigonometric functions, which can be expressed as:

$$\frac{d^2y(t)}{dt^2} + \frac{6}{t(1-t)}\frac{dy(t)}{dt} + \frac{y^2(t) + 2y(t) - 6\cosh t}{t(1-t)} - \frac{\sinh t}{t(1-t)}(\sinh t - t^2 + t + 2) = 0, \tag{19}$$

$$y(0) = 0, \quad y(1) + 0.5y\left(\frac{4}{5}\right) = \sinh 1 + 0.5\sinh\left(\frac{4}{5}\right). \tag{20}$$

With boundary conditions, the exact solution is given as: $y(t) = \sinh t$. In Eq. (19), the singularities are at $t = 0$ and $t = 1$. Now the loss for Eq. (19) is written as:

$$\mathcal{L}_f = \left| \frac{d^2y(t_i)}{dt_i^2} + \frac{6}{t_i(1-t_i))}\frac{dy(t_i)}{dt_i} + \frac{y^2(t_i) + 2y(t_i) - 6\cosh t_i}{t_i(1-t_i)} - \frac{\sinh t_i}{t_i(1-t_i)}(\sinh t_i - t_i^2 + t_i + 2) \right|^2,$$

$$\mathcal{L}_{bc} = |y(0)|^2 + \left| y(1) - \sinh(1) + 0.5y\left(\frac{4}{5}\right) - 0.5\sinh\left(\frac{4}{5}\right) \right|^2,$$

$$\mathcal{L} = \left| \frac{d^2y(t_i)}{dt_i^2} + \frac{6}{t_i(1-t_i)}\frac{dy(t_i)}{dt_i} + \frac{y^2(t_i) + 2y(t_i) - 6\cosh t_i}{t_i(1-t_i)} - \frac{\sinh t_i}{t_i(1-t_i)}(\sinh t_i - t_i^2 + t_i + 2) \right|^2 + |y(0)|^2 + \left| y(1) - \sinh(1) + 0.5y\left(\frac{4}{5}\right) - 0.5\sinh\left(\frac{4}{5}\right) \right|^2.$$

All examples considered in this study admit analytical (closed-form) solutions and are therefore used for benchmarking. Nevertheless, the proposed PINN-based framework is not restricted to problems with known exact solutions and is directly applicable to TPS–BVPs and related boundary-value problems for which no closed-form expression exists. Such unsolved cases will be the focus of future work, where PINN-predicted solutions will be validated against established numerical solvers.

In the present benchmark set, the residual formulation corresponds to a nonlinear second-order differential equation with trigonometric nonlinearities, subject to three-point boundary conditions specified at two domain boundaries and one interior point. The loss evolution across training epochs consistently demonstrates stable convergence of the proposed approach. For each test problem, the PINN, A–PINN, and regularized PINN solutions are systematically compared with the corresponding analytical solution using quantitative error metrics.

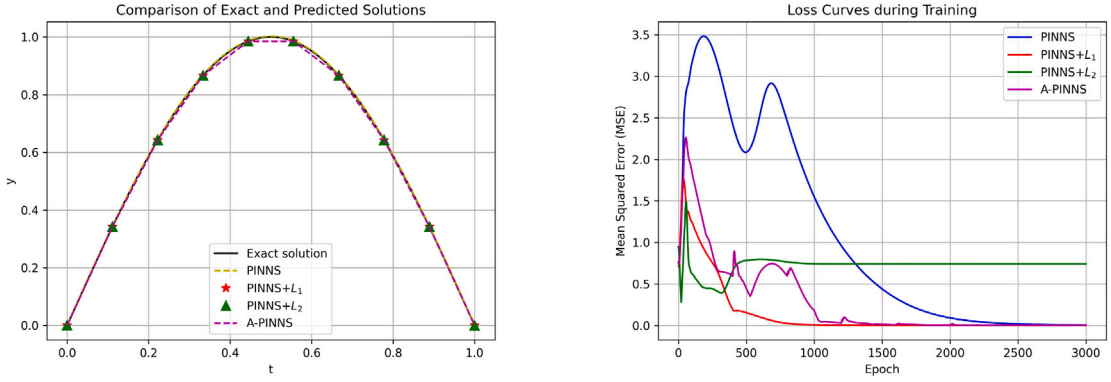
4. Results and discussion

This section presents graphical representations of all benchmark problems solved using the proposed PINN-based framework. The model was trained under four different configurations: (1) a standard PINN without regularization, (2) a PINN with L_1 regularization, denoted as PINN+L_1, (3) a PINN with L_2 regularization, denoted as PINN+L_2, and (4) an A–PINN, in which the loss-term weights were updated dynamically during training. The predicted solutions obtained from each configuration were directly compared with the corresponding analytical (true) solutions to assess the approximation accuracy, convergence behavior, and overall reliability of the learning framework.

This comparative approach provides clear insights into the influence of regularization on model performance. Additionally, the effects of different activation functions and varying hidden layer configurations on the loss behavior and convergence of each example are discussed and analyzed in detail.

Fig. 1 presents a comparative analysis between the solution obtained by the proposed PINN-based models and the exact analytical solution, along with the corresponding MSE profiles, during training. Specifically, Fig. 1(a) illustrates a comparison between the PINN-predicted solutions and the exact solution $y(t) = \sin(\pi t)$. The horizontal axis represents the independent variable $t \in [0, 1]$, and the vertical axis indicates the solution values y. The close agreement among all predicted solution curves and the analytical reference confirms the capability of the different PINN variants to accurately approximate the TPS–BVP solution. In particular, the precise matching at key domain locations, such as $t = 0$, $t = 0.5$, and $t = 1$ highlights the reliability of the neural network approximations.

The training dynamics of the networks are illustrated in Fig. 1(b), which shows the evolution of the MSE with respect to the number of training epochs. The initial fluctuations observed in the error curves



(a) (b)

Fig. 1. Comparison indicating the reliability of mode while MSE curve vs epochs give validity of results.

Table 2
Statistical analysis for Example 3.1.

Statistical metrics	PINN	PINN with L_1	PINN with L_2	A–PINN
RMSE	0.000326	0.004668	0.000157	0.000632
MAE	0.002449	0.004715	0.001260	0.005010
MAPE	0.39%	0.41%	0.92%	0.17%
R^2	0.9999	0.9998	1.0000	0.9997
R	1.0000	1.0000	1.0000	0.9998
rSD	0.9999	1.0002	0.9999	0.0062
NSE	0.9999	0.9998	1.0000	0.9996
RE	0.0039	0.0041	0.0092	0.0172

arise from the random initialization of the network weights and the transient learning phase, in which the optimizer explores the loss landscape. As the training proceeded, a general downward trend was observed across all models. Among the tested configurations, the PINN with L_1 regularization exhibited the fastest and smoothest convergence, with its MSE stabilizing very early in the training process. The A–PINN also demonstrates rapid error reduction in the early epochs; however, its loss trajectory contains noticeable oscillations before eventually stabilizing. This behavior indicates that the adaptive loss re-weighting mechanism introduces additional optimization dynamics that may delay convergence, despite ultimately achieving a low residual error. In contrast, the PINN with L_2 regularization converges more slowly and plateaus at a relatively higher error level, whereas the standard PINN gradually decreases its MSE to very small values over a longer training horizon. Overall, these results confirm that all models successfully learn the underlying solution while highlighting distinct differences in convergence speed and training stability.

The statistical analysis regarding the accuracy, consistency, and predictive reliability for Example 3.1 is summarized in Table 2. For the error-based metrics (RMSE, MAE, and RE), the standard PINN and, in particular, the PINN with L_2 regularization exhibited smaller errors than the L_1-regularized model. This indicates that the L_2 smoothness constraint is more suitable for this problem. Remarkably, the L_2-regularized PINN attained the lowest RMSE and MAE, confirming its superior approximation capability. The correlation-based statistics R^2, R, rSD, and NSE are all very close to 1 for the three classical PINN variants, which confirms an excellent agreement between the predicted and exact solutions; in this case, the PINN+L_2 configuration provides the best overall fit. The A–PINN also yielded a high level of agreement with the reference solution, as reflected by R^2, R, and NSE values close to 1, and it achieved the smallest MAPE among all models. However, its RMSE, MAE, and RE are noticeably larger than those of the standard and L_2-regularized PINNs, indicating less accurate pointwise predictions and reduced consistency. These results suggest that, for Example 3.1, the adaptive loss re-weighting mechanism does not improve the accuracy over the best-performing PINN+L_2 model and can even slightly degrade the error metrics, despite preserving a strong overall correlation with the exact solution.

For the four configurations, namely the standard PINN, the PINN with L_1 regularization (PINN+L_1), the PINN with L_2 regularization (PINN+L_2), and the A–PINN, the Fig. 2 illustrates the achieved RE across varying network depths (3, 6, 9, 12, and 15 hidden layers). The standard PINN exhibits the best performance on shallower architectures, attaining the lowest RE at depths of 3 and 6 layers. However, as the network becomes deeper, the unregularized PINN begins to degrade in performance, with noticeably higher error values observed at depths of 12 and 15 layers. This trend suggests increased susceptibility to overfitting and instability in deeper architectures when regularization is absent. In contrast, the L_1-regularized model demonstrates improved generalization capability, particularly at higher depths, owing to its sparsity-inducing effect, which helps control complexity and reduce RE. Meanwhile, the L_2-regularized model generally exhibits higher RE across most depths, with a pronounced spike at 9 layers before a modest improvement at 15 layers, indicating that L_2 regularization may not be uniformly effective for stabilizing deeper models in the present TPS-BVP setting. The A–PINN consistently records the highest RE across all network depths, indicating that adaptive re-weighting of the loss terms does not translate into accuracy gains for this problem. A plausible explanation is that the dynamic adjustment of the loss weights may over-emphasize certain residual components during training, thereby disturbing the balance required for accurate pointwise approximation. Thus, although A–PINNs remain competitive in terms of overall solution structure, they do not outperform the best classical PINN variants in terms of relative error.

As illustrated in Fig. 3, the training time for all configurations increases with the number of hidden layers, reflecting the higher computational cost associated with deeper networks. The A–PINN requires slightly more time than the standard PINN, due to the additional gradient computations associated with weight adaptation, but it remains more efficient than the regularized models. Overall, the results highlight a trade-off between accuracy and computational effort, with the best balance achieved by the classical PINN configurations.

In addition to the relative error comparison discussed earlier, Fig. 4 presents the effect of different activation functions on model accuracy using the metric $-\log_{10}(\text{RE})$, where larger values indicate smaller errors and hence superior predictive performance. Among all tested configurations, the hyperbolic tangent activation function combined with

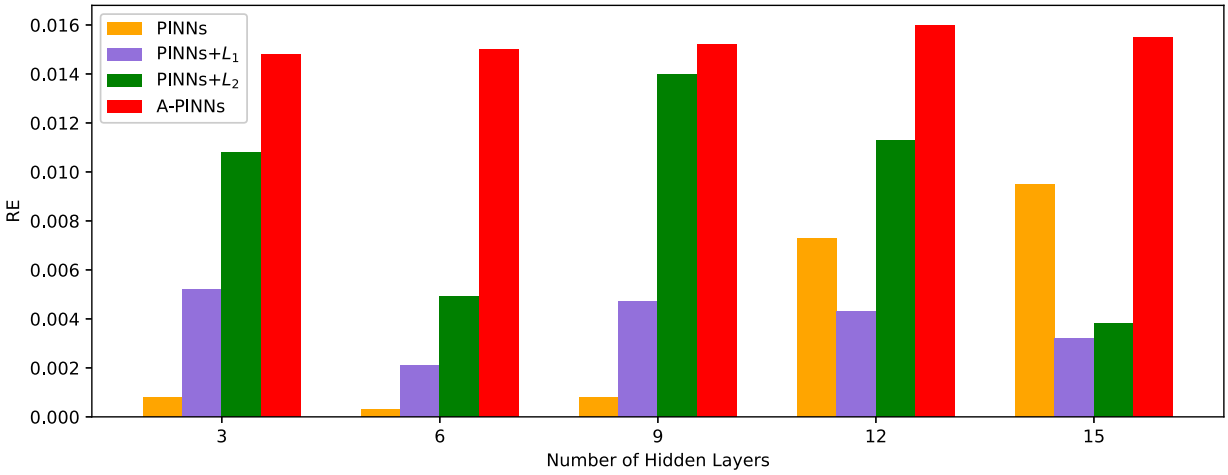


Fig. 2. Representation of Relative Errors of each training technique Vs number of layers.

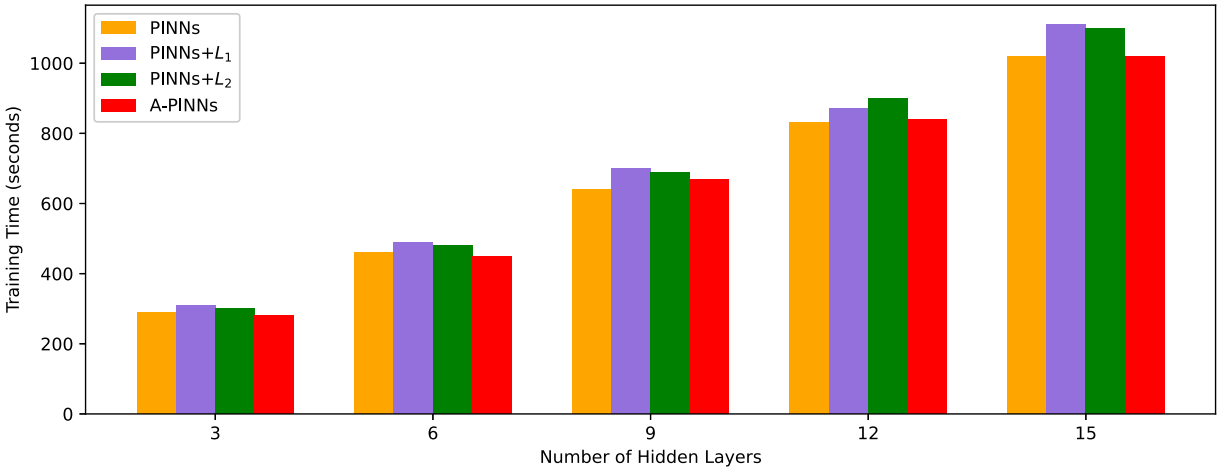


Fig. 3. Time taken by model vs number of layers.

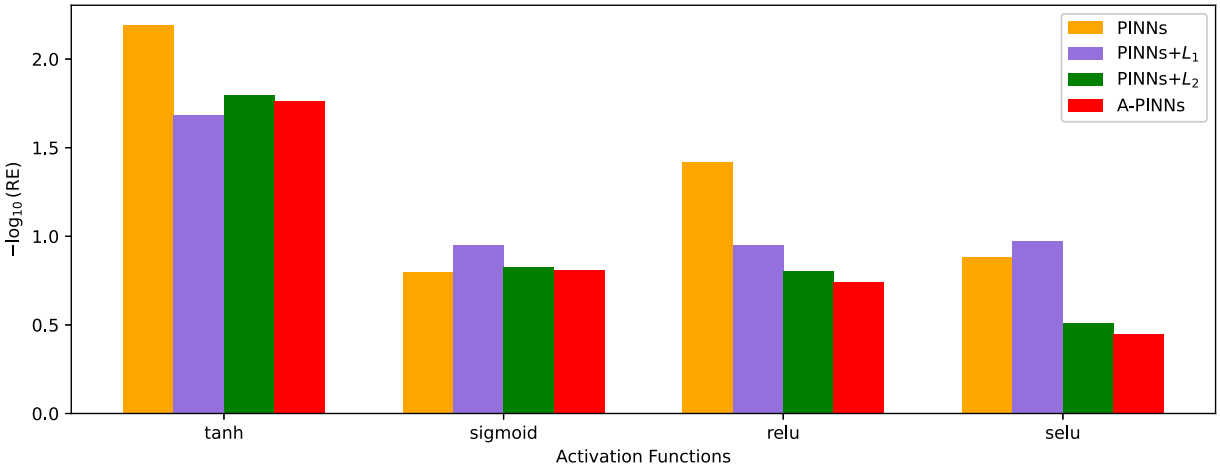


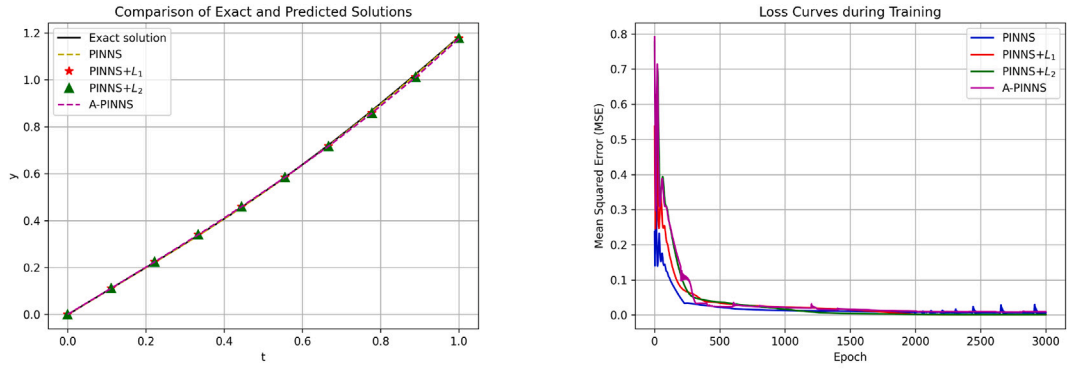
Fig. 4. Effect of various activation functions.

L_2-regularized PINNs achieves the highest accuracy, reflecting its strong ability to approximate smooth functional relationships in TPS-BVPs. The sigmoid activation function, however, exhibits the weakest performance across nearly all models, likely due to gradient saturation effects that hinder efficient optimization. ReLU activation displays moderate accuracy, whereas SeLU yields competitive performance, particularly when paired with L_1 regularization. The relatively inferior performance of L_1-regularized PINNs can be attributed to sparsity enforcement in the network weights, which may suppress small yet informative coefficients required for capturing local variations near singular points. In contrast, L_2 regularization constrains weight magnitudes while retaining descriptive model structure, thereby promoting smoother and more stable approximations. The A-PINN demonstrates consistent but not superior performance across activation functions, suggesting that adaptive loss re-weighting does not necessarily guarantee improved accuracy unless carefully tuned for the specific problem class. Overall, the results highlight that activation-function choice plays a critical role in determining the learning dynamics and accuracy of PINN architectures, and must be considered jointly with the adopted regularization strategy.

Fig. 5 presents a comparison between the approximate solution of the TPS-BVP obtained using the proposed PINN framework and the exact analytical solution, given by $y(t) = \sinh(t)$, along with the corresponding MSE curve that illustrates the training dynamics. As shown in Fig. 5(a), both the unregularized and regularized PINN models exhibit excellent agreement with the exact solution across the entire domain $t \in [0, 1]$. This strong alignment confirms the ability of the proposed model to accurately capture the underlying physical behavior dictated by the governing differential equation and its boundary conditions, even in the presence of singularities. As shown in Fig. 5(a), the A–PINNs prediction (magenta dashed curve) overlaps the exact solution $y(t) = \sinh(t)$ over $t \in [0, 1]$, indicating accurate satisfaction of the residual and three-point boundary constraints.

In the plot, the black solid line represents the exact solution, while the PINN predictions are displayed using different markers: red stars for PINN+L_1 and green triangles for PINN+L_2. The PINN+L_1 model closely follows the exact curve throughout the domain, whereas the PINN+L_2 model shows slight divergence at the beginning but aligns well with the exact solution beyond the midpoint. The near-identical behavior of all curves highlights the robustness and accuracy of the proposed PINN framework as an effective solver for singular boundary value problems.

Fig. 5(b) illustrates the evolution of the MSE over training epochs. A relatively high initial error is observed due to random initialization and inaccurate early predictions. As training progresses, the error decreases steadily, reflecting the model's learning capacity. Among all configurations, PINN+L_1 demonstrates the most stable convergence pattern, starting from a high initial value but quickly stabilizing. The standard PINN also converges smoothly toward zero error. In contrast, the PINN+L_2 model exhibits instability during the early training



(a) (b)

Fig. 5. Overlapping of PINN's and the exact curve demonstrates high performance, whereas the MSE curve converging to zero displays the accuracy.

Table 3
Statistical analysis for Example 3.2.

Statistical metrics	PINN	PINN with L_1	PINN with L_2	A-PINN
RMSE	0.004057	0.018528	0.005990	0.006374
MAE	0.004906	0.020891	0.011005	0.04705
MAPE	1.08%	5.32%	2.32%	1.61%
R^2	0.9997	0.9959	0.9988	0.9981
R	1.0000	1.0000	1.0000	0.9990
rSD	0.9843	1.0063	0.9919	0.9935
NSE	0.9997	0.9959	0.9988	0.9991
RE	0.0108	0.0532	0.0232	0.1616

phase, with noticeable error spikes. Although it eventually begins to converge, it maintains a higher loss throughout most of the training process. In Fig. 5(b), A–PINNs shows a brief transient with small oscillations due to adaptive re-weighting, followed by rapid decay of the MSE and stabilization near zero, confirming stable convergence and high accuracy.

Table 3 summarizes the predictive performance of the proposed models for Example 3.2 across multiple statistical metrics. It is apparent that the most basic version of PINN has no problem in terms of performance, as evidenced by the relatively neat and tidy errors in the figures concerning RMSE, MAE, MAPE, and RE. On the other hand, the introduction of L_2-regularization has increased the model's potential stability and predictive reliability, as evidenced by the greater values of the correlation figures R^2, R and NSE along with reasonable and manageable values of rSD. Concerning others, the version of PINN with L_1-regularization has an overwhelming high error as evidenced by RMSE = 0.0185 and MAPE = 5.32%. This explains that the sparsity-promoting regularization is not appropriate as smooth solution profiles are in order for the case of TPS-BVPs. The results are also listed for A-PINN. It is indeed facts that A-PINN has to some extent earned the rights of being called average accuracy, but this statement is scorned by the greater RE and MAE, which are the hallmark of reduced accuracy. In a nutshell, the L_2-regularized PINNs are the most accurate, smoothest and most stable, which is evidenced by the statistical data and which should be given the most credit for this type of singular three-point boundary value problems.

Fig. 6 compares the relative errors of the unregularized and regularized PINN variants across different network depths. Overall, increasing depth reduces the relative error for the standard PINN and the L_1-regularized model, indicating improved approximation capacity with deeper architectures. The L_2-regularized model follows a similar trend at moderate depths, but exhibits a noticeable increase at 15 hidden layers, suggesting sensitivity to over-regularization or optimization stiffness in deeper networks. In contrast, A–PINNs yields the largest relative errors for all depths considered, implying that the adaptive re-weighting strategy does not provide an advantage for this smooth benchmark and may introduce additional optimization dynamics that hinder accuracy in terms of relative error. Among all configurations, PINN + L_1 achieves the lowest relative errors across depths, followed closely by the

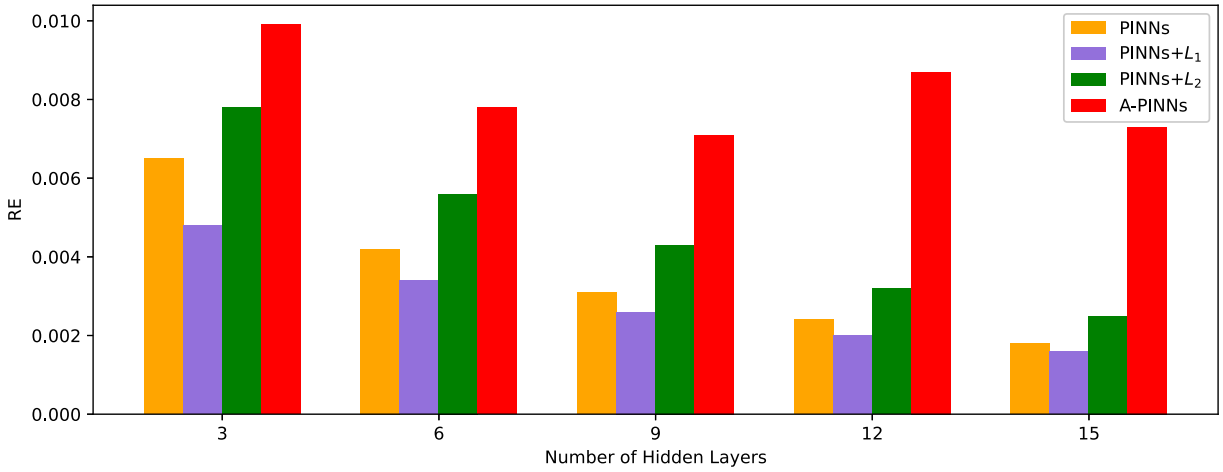


Fig. 6. Visual representation of relative errors for original and regularization techniques vs number of layers.

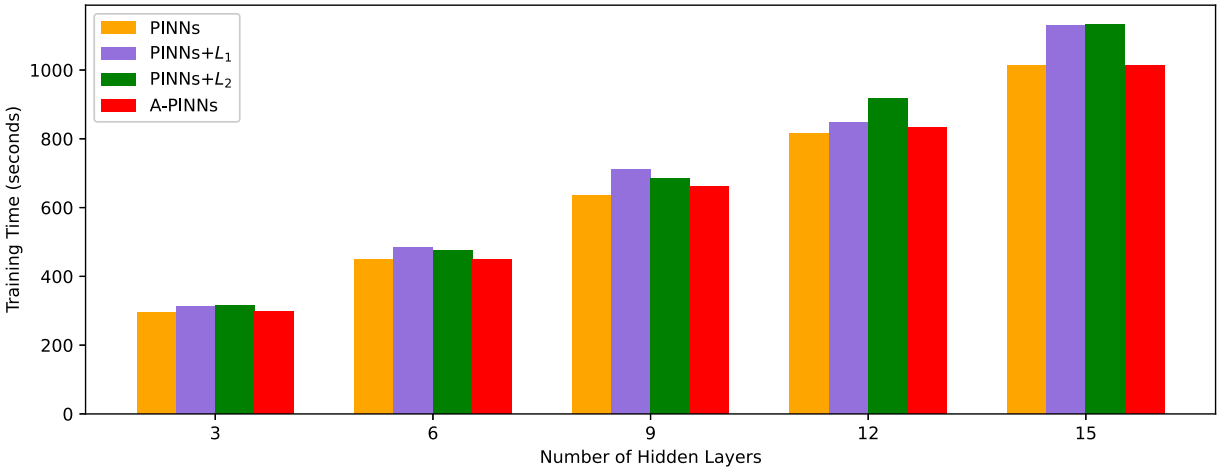


Fig. 7. Time consumption for each method vs number of layers.

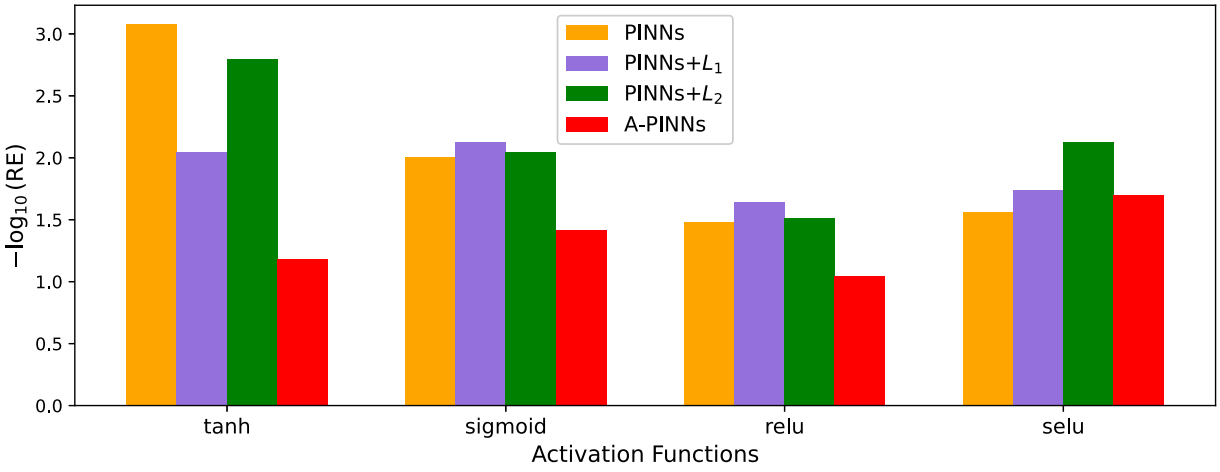


Fig. 8. Activation function vs RE.

standard PINN, whereas PINN + L_2 and A–PINNs are comparatively less competitive for deeper settings.

Fig. 7 illustrates the dependence of training time on network depth for all model variants. As expected, the computational cost increases monotonically with the number of hidden layers due to the larger parameter space and the higher cost of backpropagation. While PINN+L_1 and PINN+L_2 generally require longer runtimes than the baseline PINN, the A–PINN configuration remains competitive and is often comparable to, or slightly faster than, the unregularized PINN at moderate depths (e.g., 6–12 layers). At the largest depth (15 layers), A–PINN again exhibits the lowest training time among the tested approaches, indicating that adaptive loss re-weighting does not introduce a prohibitive overhead in practice. Overall, these results show that deeper architectures increase runtime for all methods, while A–PINNs can retain favorable computational efficiency alongside the regularized and baseline formulations.

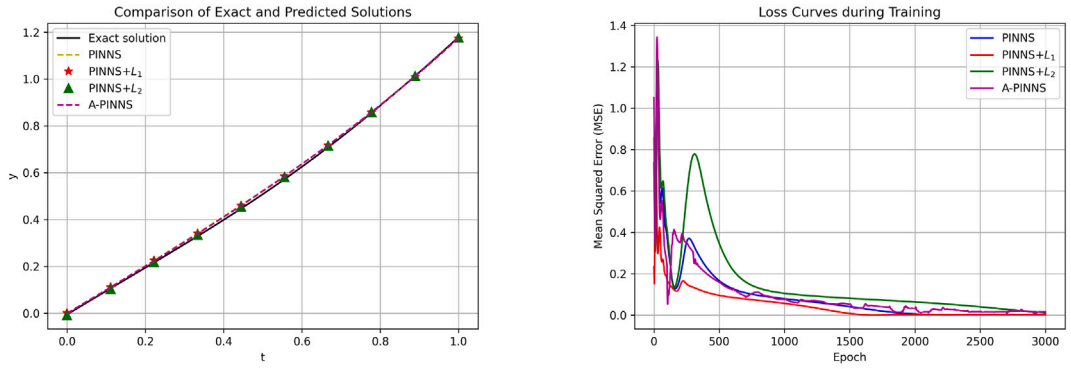
Fig. 8 illustrates the performance of various activation functions using the negative base-10 logarithm of the relative error, $-\log_{10}(\text{RE})$, as the evaluation metric. This logarithmic transformation is particularly useful for enhancing the visibility of small error differences, thereby enabling a more refined comparison of model accuracy. In this figure, taller bars correspond to lower relative errors, indicating higher predictive accuracy.

Among the tested activation functions, `tanh` and `SeLU` yield the highest $-\log_{10}(\text{RE})$ values, suggesting superior performance in minimizing error. The `ReLU` function demonstrates moderate accuracy, whereas `sigmoid` consistently results in the lowest values across all configurations. The near-zero values associated with the `sigmoid` activation imply poor model performance and insufficient learning capability in the context of the TPS-BVP considered.

Fig. 9 compares the solution obtained using the proposed unregularized PINN model with those of the regularized variants and the exact analytical solution over the domain $t \in [0, 1]$. It also includes the corresponding mean squared error (MSE) curves illustrating training progression over epochs.

In Fig. 9(a), the horizontal axis represents the independent variable t, while the vertical axis denotes the solution values $y(t)$. The black solid line corresponds to the exact analytical solution. The predicted solutions from different models are plotted using colored markers: red stars for PINN+L_1 and green triangles for PINN+L_2. The unregularized PINN closely follows the exact curve across the domain, particularly in regions exhibiting singular behavior. The PINN+L_1 model also tracks the exact solution well, while the PINN+L_2 model shows slight divergence near the start of the interval but aligns well in the latter half. The A–PINNs prediction (magenta dashed curve) also closely overlaps the exact analytical solution throughout $t \in [0, 1]$, indicating that the adaptive loss re-weighting preserves accurate satisfaction of both the residual and the three-point boundary constraints.

The near-complete overlap of the predicted and exact solution curves demonstrates the accuracy and robustness of the proposed framework



(a) (b)

Fig. 9. Visual representation of results generated by regularized and unregularized PINN model and loss curves for Example 3.3.

Table 4
Statistical analysis for Example 3.3.

Statistical metrics	PINN	PINN with L_1	PINN with L_2	A–PINN
RMSE	0.001070	0.000957	0.000893	0.007987
MAE	0.007509	0.001319	0.001164	0.007101
MAPE	1.02%	0.23%	1.16%	4.81%/
R^2	0.9995	1.0000	1.0000	0.99941
R	1.0000	1.0000	1.0000	0.9999
rSD	1.0014	1.0033	1.0027	0.00946
NSE	0.9995	1.0000	1.0000	0.99941
RE	0.0102	0.0023	0.0116	0.0481

in capturing the physical behavior of the system, even without large labeled datasets. These results confirm that the PINN model, both regularized and unregularized, can effectively handle singularities and complex boundary conditions.

Fig. 9(b) illustrates the evolution of the MSE throughout training. All models exhibit relatively large initial errors due to random weight initialization and early prediction inaccuracies. A steep reduction in error is observed during the initial training phase for all models. The unregularized PINN (blue line) achieves the fastest and smoothest convergence, reaching a minimal error level earlier than the regularized variants. However, this may suggest that the model is trapped in a local optimum rather than generalizing optimally. In contrast, the PINN+L_2 model (green) starts with a higher error but decreases steadily, indicating improved generalization and reduced risk of overfitting. The PINN+L_1 model (red) initially fluctuates but quickly stabilizes and converges toward zero error, mirroring the behavior of the other configurations. A–PINNs exhibits a short transient phase with noticeable oscillations in the MSE due to the continuous adjustment of the adaptive weights; however, the error subsequently decays and stabilizes near zero, confirming reliable convergence and accuracy comparable to the other PINN variants.

The observed behavior confirms that the PINN framework successfully captures the physics embedded in the problem and yields reliable, accurate solutions for the considered TPS-BVP. The optimal MSE values were achieved by systematically tuning the network hyperparameters, as summarized in Table 5. The solution profiles shown in the preceding figures were generated using these optimized configurations, which minimized the MSE. Lower MSE values reflect the superior predictive capability of the model and its effectiveness in approximating the nonlinear characteristics of the underlying boundary value problem.

The consolidated results in Table 4 confirm the strong performance of the unregularized and regularized PINN variants. Across all test cases, the PINN-based models yield consistently low RMSE, MAE, and MAPE values, demonstrating accurate recovery of the benchmark solutions. Among the regularization strategies, the standard L_1-regularized PINN provides a modest performance improvement relative to the baseline PINN, indicating that mild sparsity promotion can be beneficial when the target solution is smooth and well-behaved. The complementary indicators rSD and NSE further support the reliability of the proposed formulations, showing stable error dispersion and strong agreement with the reference solution. In contrast, the A–PINN configuration exhibits comparatively weaker performance and is consistently outperformed by the standard and regularized PINNs, suggesting that adaptive re-weighting does not offer an advantage for the present problem settings. Overall, the results indicate that regularization, particularly L_2 can enhance predictive reliability and robustness.

It is also noteworthy that both regularized models attained perfect values for the coefficient of determination (R^2 = 1.0000) and the NSE = 1.0000, indicating excellent agreement between predicted and true solutions.

While the unregularized (baseline) PINN performed well, its accuracy was slightly lower compared to the regularized models. The PINN+L_2 configuration also demonstrated strong performance, though with slightly higher MAE and RE values than the L_1-based model. These findings suggest that for the considered TPS-BVP, the PINN+L_1 model offers the optimal trade-off between bias and generalization, making it the most accurate and reliable configuration among those tested. In contrast, A–PINNs delivers substantially poorer accuracy (RMSE = 0.007987, MAE = 0.007101, MAPE = 4.81%, RE = 0.0481) and slightly lower goodness-of-fit (R^2 = 0.99941; NSE = 0.99941), confirming that the adaptive weighting strategy is not competitive for Example 3.3 relative to the baseline and regularized PINNs.

The bar graph illustrating the RE for all model configurations is presented in Fig. 10. The PINN+L_1 model consistently yields the lowest RE values across all network depths, particularly as model complexity increases. The unregularized PINN also maintains comparatively low RE values, indicating stable performance with increasing depth. In contrast, the A–PINNs configuration exhibits substantially larger RE values at every depth, with a pronounced increase at 12 hidden layers, suggesting that the adaptive re-weighting strategy does not translate into improved relative-error performance for this case. Overall, these results demonstrate that the PINN+L_1 configuration is the most effective in minimizing prediction error and is therefore the best-suited model for the given TPS–BVP.

Furthermore, Fig. 11 presents the training time required for each model across varying numbers of hidden layers. As expected, the training time increases monotonically with network depth for all configurations due to the larger number of trainable parameters. The standard (unregularized) PINN consistently achieves the lowest training time across the tested depths, indicating superior computational efficiency relative to the regularized variants. In contrast, the A–PINNs configuration

Table 5
Effect of training configuration (epochs, activation function, and network width) on the mean squared error (MSE) for three benchmark examples. Results are shown for PINNs, PINNs + L_1, PINNs + L_2, and A-PINNs under identical hyperparameter settings.

Epochs	Activation	Hidden Neurons	PINNs (MSE)			PINNs + L_1 (MSE)			PINNs + L_2 (MSE)			A-PINNs (MSE)		
			Exp. 1	Exp. 2	Exp. 3	Exp. 1	Exp. 2	Exp. 3	Exp. 1	Exp. 2	Exp. 3	Exp. 1	Exp. 2	Exp. 3
5000	tanh	[20,20,20]	0.218824	0.002053	0.009721	0.192392	0.001895	0.008119	0.170529	0.001680	0.007197	0.142108	0.001400	0.005997
5000	tanh	[30,30,30]	0.322930	0.007213	0.002319	0.281962	0.006613	0.001924	0.252339	0.005918	0.001721	0.212549	0.004985	0.001450
5000	tanh	[50,50,50]	0.386446	0.005057	0.006640	0.334620	0.004598	0.005462	0.284666	0.003911	0.004647	0.242485	0.003332	0.003958
5000	selu	[20,20,20]	0.080107	0.018193	0.004628	0.074133	0.017678	0.004069	0.068024	0.016221	0.003733	0.056475	0.013467	0.003100
5000	selu	[30,30,30]	0.070599	0.017355	0.012716	0.064873	0.016745	0.011100	0.056571	0.014602	0.009680	0.050427	0.013016	0.008629
5000	selu	[50,50,50]	0.071193	0.008649	0.001811	0.065488	0.008153	0.001654	0.057715	0.007288	0.001381	0.048485	0.006185	0.001172
5000	relu	[20,20,20]	0.779014	0.023888	0.717854	0.760353	0.024815	0.694106	0.709941	0.023022	0.646807	0.625129	0.020341	0.562414
5000	relu	[30,30,30]	0.510609	0.017485	0.346871	0.495091	0.018188	0.333580	0.447145	0.016498	0.302380	0.410205	0.015025	0.277303
5000	relu	[50,50,50]	0.599544	0.011970	0.088490	0.550392	0.011226	0.083980	0.513275	0.010511	0.078431	0.466142	0.009573	0.071030
5000	sigmoid	[20,20,20]	1.467156	0.255956	0.602457	1.201271	0.227963	0.467085	1.040022	0.199513	0.407344	0.884788	0.173436	0.351812
5000	sigmoid	[30,30,30]	0.436951	0.223094	0.397432	0.358356	0.198761	0.308439	0.301536	0.173892	0.267858	0.256828	0.150593	0.229718
5000	sigmoid	[50,50,50]	1.525069	0.231441	0.249954	1.227773	0.205649	0.190042	1.015706	0.179716	0.159690	0.894901	0.156665	0.137116
10,000	tanh	[20,20,20]	0.210464	0.003853	0.005267	0.191493	0.003686	0.004294	0.165361	0.003248	0.003774	0.136135	0.002598	0.003028
10,000	tanh	[30,30,30]	0.161954	0.002263	0.008855	0.139526	0.002152	0.007160	0.121140	0.001884	0.006290	0.101385	0.001507	0.005219
10,000	tanh	[50,50,50]	0.132757	0.001140	0.001970	0.109167	0.000968	0.001513	0.094378	0.000867	0.001355	0.075904	0.000715	0.001118
10,000	selu	[20,20,20]	0.054094	0.011599	0.007563	0.051361	0.011764	0.006770	0.046359	0.010512	0.006084	0.037891	0.008635	0.004998
10,000	selu	[30,30,30]	0.038798	0.008781	0.002166	0.035798	0.008842	0.001922	0.031001	0.007831	0.001681	0.025007	0.005943	0.001326
10,000	selu	[50,50,50]	0.042377	0.003194	0.002118	0.037336	0.003041	0.001792	0.032963	0.002712	0.001566	0.027225	0.002154	0.001254
10,000	relu	[20,20,20]	0.752730	0.010215	1.456074	0.738560	0.010529	1.369680	0.681761	0.009429	1.258265	0.582068	0.008201	1.090835
10,000	relu	[30,30,30]	0.286082	0.009903	0.696064	0.264598	0.009626	0.621120	0.246739	0.008722	0.565241	0.203842	0.007410	0.479585
10,000	relu	[50,50,50]	0.042232	0.005027	0.350766	0.038125	0.004929	0.313173	0.034889	0.004412	0.284812	0.028997	0.003723	0.243461
10,000	sigmoid	[20,20,20]	0.388610	0.138549	0.296871	0.314182	0.127036	0.220978	0.276559	0.110919	0.193701	0.230289	0.096624	0.164986
10,000	sigmoid	[30,30,30]	0.211389	0.119513	0.210334	0.165417	0.107077	0.155168	0.146845	0.093102	0.137355	0.122400	0.080875	0.116802
10,000	sigmoid	[50,50,50]	1.088798	0.011036	0.134353	0.859713	0.010268	0.098170	0.737813	0.008889	0.082622	0.606310	0.006453	0.071075
15,000	tanh	[20,20,20]	0.198540	0.003120	0.004612	0.177882	0.002986	0.003781	0.154311	0.002640	0.003312	0.125904	0.002110	0.002654
15,000	tanh	[30,30,30]	0.149280	0.001865	0.007920	0.126915	0.001742	0.006410	0.111204	0.001530	0.005641	0.092338	0.001220	0.004690
15,000	tanh	[50,50,50]	0.121440	0.000960	0.001650	0.098312	0.000812	0.001270	0.085901	0.000731	0.001140	0.069775	0.000600	0.000950
15,000	selu	[20,20,20]	0.047820	0.010520	0.006840	0.045410	0.010710	0.006110	0.041220	0.009560	0.005520	0.033500	0.007820	0.004510
15,000	selu	[30,30,30]	0.034950	0.007920	0.001980	0.032140	0.007980	0.001740	0.027820	0.007050	0.001520	0.022430	0.005340	0.001180
15,000	selu	[50,50,50]	0.038010	0.002650	0.001890	0.033220	0.002510	0.001600	0.029410	0.002250	0.001410	0.024350	0.001780	0.001120
15,000	relu	[20,20,20]	0.703400	0.009820	1.320100	0.688900	0.010010	1.241800	0.634700	0.008950	1.141200	0.542200	0.007780	0.990400
15,000	relu	[30,30,30]	0.262900	0.009340	0.631500	0.241700	0.009080	0.564800	0.226100	0.008240	0.514900	0.186700	0.006980	0.436200
15,000	relu	[50,50,50]	0.038900	0.004620	0.312500	0.035100	0.004520	0.279600	0.032100	0.004050	0.254200	0.026700	0.003410	0.217900
15,000	sigmoid	[20,20,20]	0.342800	0.126900	0.269400	0.276900	0.116400	0.201100	0.243500	0.101800	0.176200	0.203800	0.088400	0.149800
15,000	sigmoid	[30,30,30]	0.187600	0.109600	0.192700	0.145700	0.098000	0.142300	0.129900	0.085300	0.126100	0.108200	0.073800	0.107700
15,000	sigmoid	[50,50,50]	0.965500	0.009850	0.121100	0.755900	0.009120	0.089200	0.653400	0.007980	0.075100	0.540800	0.005920	0.064100
20,000	tanh	[20,20,20]	0.186900	0.002650	0.004020	0.166500	0.002520	0.003310	0.143800	0.002220	0.002890	0.116700	0.001760	0.002280
20,000	tanh	[30,30,30]	0.137200	0.001520	0.007180	0.115900	0.001410	0.005780	0.101600	0.001240	0.005090	0.083900	0.000980	0.004220
20,000	tanh	[50,50,50]	0.112300	0.000780	0.001420	0.090600	0.000660	0.001080	0.079200	0.000595	0.000980	0.064900	0.000490	0.000800
20,000	selu	[20,20,20]	0.042900	0.009610	0.006120	0.040800	0.009780	0.005470	0.037000	0.008720	0.004960	0.030100	0.007120	0.004050
20,000	selu	[30,30,30]	0.031200	0.007210	0.001760	0.028700	0.007260	0.001540	0.024900	0.006420	0.001340	0.020100	0.004850	0.001050
20,000	selu	[50,50,50]	0.034700	0.002210	0.001720	0.030400	0.002090	0.001450	0.026900	0.001870	0.001280	0.022400	0.001480	0.001010
20,000	relu	[20,20,20]	0.662100	0.009410	1.198400	0.647300	0.009590	1.128600	0.595800	0.008560	1.037200	0.509200	0.007430	0.900500
20,000	relu	[30,30,30]	0.241800	0.008880	0.575200	0.221200	0.008640	0.514600	0.207000	0.007860	0.470100	0.171000	0.006650	0.399200
20,000	relu	[50,50,50]	0.035400	0.004210	0.284900	0.031900	0.004110	0.255200	0.029200	0.003690	0.232700	0.024300	0.003100	0.199500
20,000	sigmoid	[20,20,20]	0.309500	0.115400	0.242800	0.249800	0.105900	0.181600	0.220500	0.092800	0.159400	0.185600	0.080700	0.135900
20,000	sigmoid	[30,30,30]	0.169200	0.099900	0.176400	0.131600	0.089300	0.130100	0.117700	0.077700	0.115400	0.098400	0.067000	0.098600
20,000	sigmoid	[50,50,50]	0.881300	0.008820	0.109700	0.690200	0.008150	0.080900	0.597800	0.007120	0.068300	0.494600	0.005260	0.058200

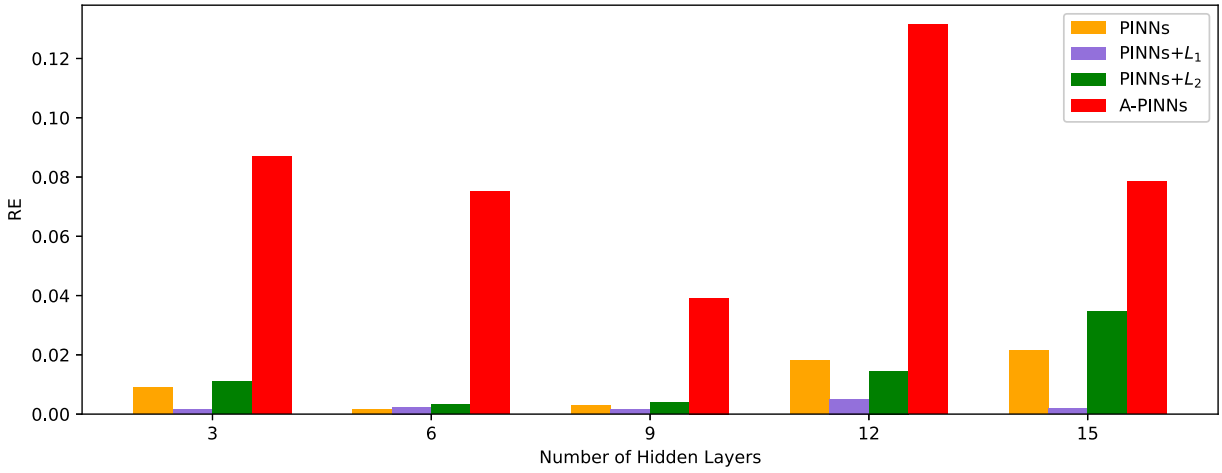


Fig. 10. Relative error vs number of layers.

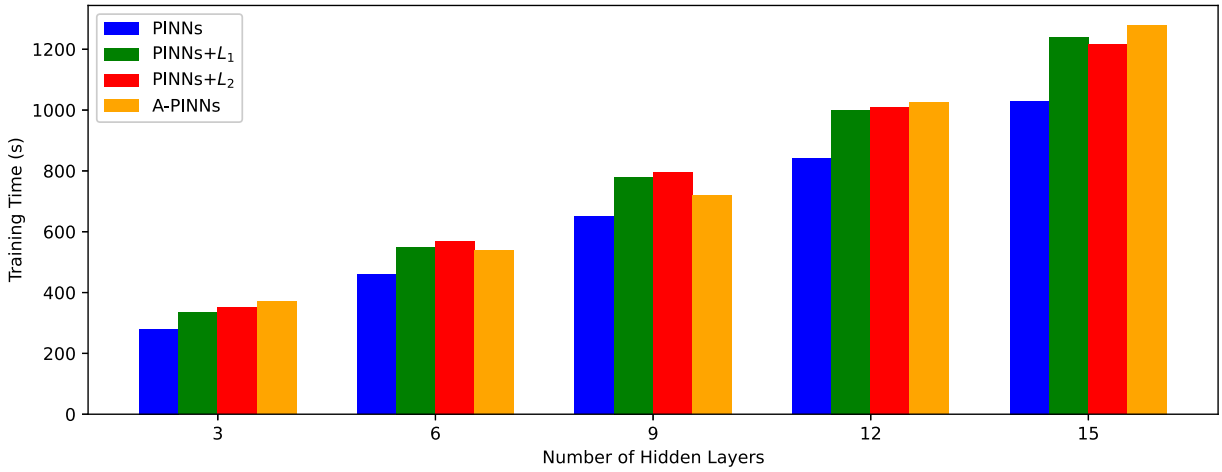


Fig. 11. Time consumption for each technique vs number of layers.

incurs the highest computational cost, particularly at larger depths (e.g., 12 and 15 layers), reflecting the additional overhead associated with adaptive loss re-weighting during optimization. Overall, these results highlight that A–PINNs does not provide a runtime advantage in this setting and that the baseline PINN remains the most time-efficient choice. The proposed constrained regularization still achieves faster convergence than traditional hybrid models, such as GNN-based schemes, which typically require multiple derivative evaluations. Although training time generally increases with network depth, the favorable balance between computational cost and accuracy achieved by the proposed PINN framework supports its practical deployment.

The bar chart in Fig. 12 compares the performance of various activation functions using the metric $-\log_{10}(\text{RE})$, which serves as a reliable indicator of model accuracy. This negative logarithmic transformation reverses the scale, such that higher values of $-\log_{10}(\text{RE})$ correspond to lower relative errors, and lower values indicate higher prediction errors.

As shown in the figure, the hyperbolic tangent (`tanh`) activation function yields the highest $-\log_{10}(\text{RE})$ values, particularly for the `PINN`+L_2 configuration, indicating superior performance in reducing relative error. In contrast, the sigmoid activation function consistently produces the lowest bars across all models, signifying poorer predictive accuracy. The `SeLU` and `ReLU` functions follow a similar trend to `tanh`, producing relatively high $-\log_{10}(\text{RE})$ values, which demonstrates their consistent ability to reduce relative error.

Table 5 further highlights the influence of activation function choice and network architecture on model performance. For Example 3.1, the best performance was achieved using the `SeLU` activation function with a network architecture of [30, 30, 30] neurons trained over 10,000 epochs, resulting in the lowest MSE of 0.038798. In Example 3.2, the optimal configuration involved the `tanh` activation function with a deeper architecture of [50, 50, 50] neurons trained over 10,000 epochs, yielding an MSE of 0.001140. Similarly, for Example 3.3, the best performance was obtained using the `SeLU` activation function with a [50, 50, 50] architecture trained over 5000 epochs, resulting in an MSE of 0.001811.

These results demonstrate that the careful selection of activation functions, network depth, and training duration plays a critical role in enhancing the predictive performance of the PINN framework. The strong and consistent performance of the `SeLU` and `tanh` activation functions in minimizing MSE across all test cases underscores their suitability for modeling the complex and singular behavior of TPS-BVPs.

The present study emphasizes the standard formulation of a PINN, although a conceptual comparison with more recent variants like A-PINNs and Physics-Informed Extreme Learning Machines (PIELM) may also be interesting. While A-PINNs focus on collocation points and dynamically adjust them to reduce a problem's residuals, it is important to note that with singular problems, increasing computational complexity is of much more concern. On the other hand, PIELM removes the iterative weight updating process, and consequently, convergence is accelerated

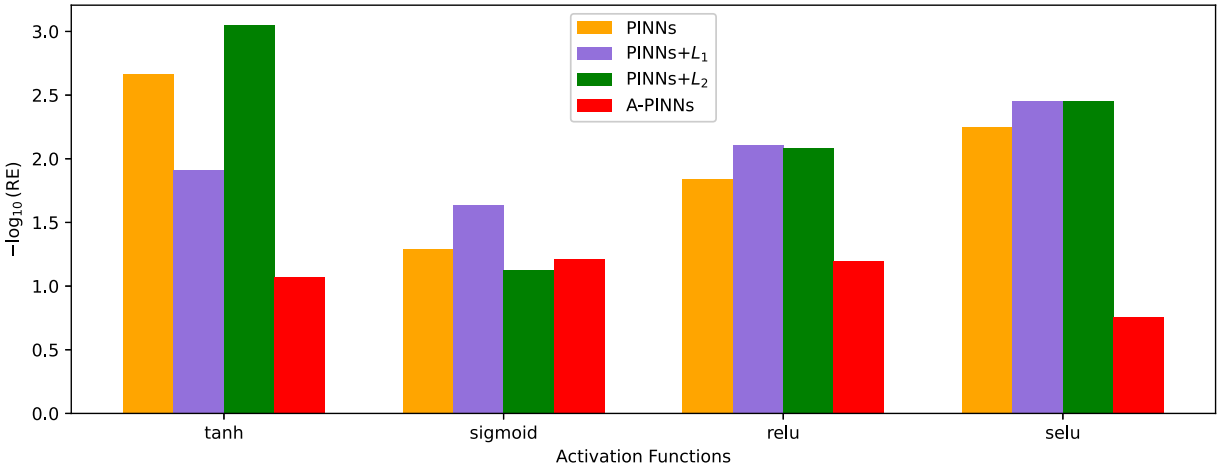


Fig. 12. Activation function vs RE.

to the potential detriment of flexibility for extremely nonlinear problems. In this case, the proposed PINN framework is more advantageous for PIELM's targeted simplicity, as it upholds PIELM's computational simplicity while maintaining the expansion of convergence to many singular problems. The lack of adaptive tuning, architecture variance, and other modifications also demonstrate the practicality and robustness of the proposed method for TPS-BVPs with the achieved mean squared errors.

5. Conclusion

This study investigated the effectiveness of PINNs for solving nonlinear dynamical systems, with a particular emphasis on TPS–BVPs. The proposed framework incorporates the governing physical laws directly into the training process, enabling the model to accurately approximate solutions by minimizing the residuals of the differential equations and satisfying the prescribed boundary conditions, without requiring labeled data. The model's performance was validated through comparisons with exact analytical solutions, demonstrating excellent agreement and low MSE values across all test cases. A systematic analysis of various architectural and training parameters, including activation functions, network depth, and number of training epochs, revealed that appropriate tuning significantly enhances model performance. Importantly, the study also highlighted the benefits of incorporating regularization techniques, specifically L_1 and L_2 regularization, into the PINN framework. Regularization was shown to improve generalization, control model complexity, and mitigate overfitting, particularly in deeper networks. Among the configurations tested, the `PINN`+L_1 model achieved the most accurate results in several cases, indicating that constrained optimization plays a key role in improving the robustness of PINN-based solvers for singular problems.

In addition, the A-PINN formulation was examined as an adaptive alternative that dynamically re-weights the residual and constraint losses during training. While A-PINNs produced accurate solution profiles in several benchmarks, its quantitative performance was not consistently competitive with the baseline and regularized PINNs and, in some cases, exhibited increased relative errors and less stable transient training dynamics. These observations suggest that adaptive loss re-weighting may be problem-dependent and requires careful tuning to provide systematic benefits for the TPS-BVP settings considered.

Overall, the findings confirm that the proposed PINN methodology provides a reliable, flexible, and computationally efficient alternative to traditional numerical methods for solving singular boundary value problems. Its success across multiple benchmark examples suggests strong potential for application to more complex, higher-dimensional, and highly nonlinear systems. The proposed approach is probably useful for the modeling of physical systems described by three-point or multipoint boundary conditions, especially systems in diffusion, heat transfer, and reaction–diffusion. Future work will focus on extending this approach to multi-physics problems and real-world engineering systems with more challenging boundary value problems, particularly high-order, multidimensional problems, as well as problems with time-dependent singular systems. Including adaptive collocation, multi-fidelity learning, as well as hybrid optimizers, particularly PIELM and reinforcement-learning-assisted PINNs, is likely to improve convergence within the proposed time and accuracy constraints. In addition, focusing on problems involving heat transfer and fluid–structure interaction applied to experimental or industrial data will be critical in evaluating the method's practical effectiveness and scalability. When moving into more advanced multi-physics and higher-dimensional PDEs, two principal issues become apparent, the higher-order derivative computations become expensive in memory and operations and unstable gradients during training. We will pursue scalable variations of PINNs based more on sparse sampling, domain decomposition, and parallel computation of gradients. This can also be addressed in the future.

CRediT authorship contribution statement

Uzma Nadeem: Writing – original draft, Formal analysis, Data curation. **Malik Ahsin Iqbal:** Writing – review & editing, Formal analysis, Conceptualization. **Sadique Rehman:** Writing – review & editing, Formal analysis, Data curation. **Aamir Farooq:** Writing – review & editing, Supervision, Software, Conceptualization. **Hijaz Ahmad:** Writing – review & editing, Methodology, Funding acquisition, Conceptualization. **Fuad A. Awwad:** Supervision, Software, Methodology. **Emad A.A. Ismail:** Validation, Project administration, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors gratefully acknowledge the financial support provided by the Ongoing Research Funding Program (ORF-2026-1060), King Saud University, Riyadh, Saudi Arabia.

Data availability statement

The data supporting this study's findings are available in the article.

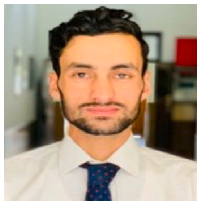
References

[1] Abbasi J, Andersen PØ. Physical activation functions (pafs): an approach for more efficient induction of Physics into physics-informed neural networks (pinns). Neurocomputing 2024;608:128352.
[2] Abdi GR, Sheikhani AHR, Kordrostami S, Zarei B, Rad MF. Identifying communities in complex networks using learning-based genetic algorithm. Ain Shams Eng J 2024;15(12):103031.
[3] Abu Arqub O, Rashaideh H. The rkhs method for numerical treatment for integrodifferential algebraic systems of temporal two-point bvps. Neural Comput Appl 2018;30:2595–606.
[4] Agarwal RP, O'Regan D, Rachůnková I, Staněk S. Two-point higher-order bvps with singularities in phase variables. Comput Math Appl 2003;46(12):1799–826.
[5] Apicella A, Donnarumma F, Isgrò F, Prevete R. A survey on modern trainable activation functions. Neural Networks 2021;138:14–32.
[6] Benesty J, Chen J, Huang Y, Cohen I. Pearson correlation coefficient. In: Noise reduction in speech processing. Springer; 2009. pp. 1–4.
[7] Bischof R, Kraus MA. Multi-objective loss balancing for physics-informed deep learning. Comput Methods Appl Mech Eng 2025;439:117914.
[8] Bock S, Goppold J, Weiß M. An improvement of the convergence proof of the adam-optimizer. [arXiv preprint] arXiv:1804.10587. 2018.
[9] Braslow AL. A review of factors affecting boundary-layer transition. 1966.
[10] Chakraverty S, Mall S. Single layer Chebyshev neural network model with regression-based weights for solving nonlinear ordinary differential equations. Evolutionary Intelligence 2020;13(4):687–94.
[11] Costabal FS, Pezzuto S, Perdikaris P. δ-pinns: physics-informed neural networks on complex geometries. Eng Appl Artif Intell 2024;127:107324.
[12] Cuong DV, Lalić B, Petrić M, Nguyen B, Roantree M. Adapting physics-informed neural networks to optimize odes in mosquito population dynamics. [arXiv preprint] arXiv:2406.05108. 2024.
[13] De Florio M, Schiassi E, Calabrò F, Furfaro R. Physics-informed neural networks for 2nd order ODES with sharp gradients. J Comput Appl Math 2024;436:115396.
[14] De Myttenaere A, Golden B, Le Grand B, Rossi F. Mean absolute percentage error for regression models. Neurocomputing 2016;192:38–48.
[15] Di Bucchianico A. Coefficient of determination (r 2). In: Encyclopedia of statistics in quality and reliability. Hoboken, NJ: John Wiley & Sons, Ltd.; 2008. ISBN 978-0-470-06157-2.
[16] Mean absolute error. Available online at: https://en.wikipedia.org/wiki/Mean_absolute_error (Accessed: 19 Sep 2026).
[17] Evans LC. Partial differential equations, vol. 19. American Mathematical Society; 2022.
[18] Farda I, Thammano A. An adaptive differential evolution with multiple crossover strategies for optimization problems. HighTech Innov J 2024;5(2):231–58.
[19] Feng Y, Li L, Liu Q. A kernel loss for solving the Bellman equation. Adv Neural Inf Process Syst 2019;32.
[20] Geng F. Solving singular second order three-point boundary value problems using reproducing kernel hilbert space method. Appl Math Comput 2009;215(6):2095–102.
[21] Greenspan HP. Models for the growth of a solid tumor by diffusion. Stud Appl Math 1972;51(4):317–40.
[22] Gries ST, Ellis NC. Statistical measures for usage-based linguistics. Language Learning 2015;65(S1):228–55.
[23] Guo J, Zhu H, Yang Y, Guo C. Advances in physics-informed neural networks for solving complex partial Differential equations and their engineering applications: a systematic review. Eng Appl Artif Intell 2025;161:112044.
[24] Gupta CP. Solvability of a three-point nonlinear boundary value problem for a second order ordinary differential equation. J Math Anal Appl 1992;168(2):540–51.
[25] Hartman P. Ordinary differential equations. SIAM; 2002.
[26] Hodson TO. Root mean square error (RMSE) or mean absolute error (MAE): when to use them or not. Geosci Model Dev Discuss 2022;2022:1–10.
[27] Huo L, Wei S, Wang I. Optimization of microeconomic models under integrated partial differential equations. HighTech Innov J 2024;5(4):995–1007.
[28] Il'in VA, Moiseev EI. A nonlocal boundary value problem of the first kind for the sturm–liouville operator in differential and difference interpretations. Differentsialnye Uravneniya 1987;23(7):1198–207.
[29] Keller JB. Electrohydrodynamics i. The equilibrium of a charged gas in a container. J Ration Mech Anal 1956:715–24.
[30] Kervadec H, Bouchtiba J, Desrosiers C, Granger E, Dolz J, Ayed IB. Boundary loss for highly unbalanced segmentation. In: International Conference on medical Imaging with deep learning. PMLR; 2019. p. 285–96.
[31] Lei Y-J, Wang B-Y, Yang Y-T. Optimizing the loss function for bounding box regression through scale smoothing. Ain Shams Eng J 2024;15(11):103046.
[32] Lu B, Moya C, Lin G. Nsga-Pinn: a multi-objective optimization method for physics-informed neural network training. Algorithms 2023;16(4):194.
[33] Manteuffel TA, White AB. The numerical solution of second-order boundary value problems on nonuniform meshes. Math Comput 1986;47(176):511–35.
[34] McClenny L, Braga-Neto U. Self-adaptive physics-informed neural networks. [arXiv preprint] arXiv:2009.04544. 2020.
[35] McCuen RH, Knight Z, Cutter AG. Evaluation of the nash–sutcliffe efficiency index. J Hydrol Eng 2006;11(6):597–602.
[36] Pandey RK, Barnwal AK. Multiple solutions for nonlinear doubly singular three-point boundary value problems with derivative dependence. Int J Differ Equations 2012;2012(1):838947.
[37] Park H, Stefanski LA. Relative-error prediction. Stat Probab Lett 1998;40(3):227–36.
[38] Parsons HM, Ekman DR, Collette TW, Viant MR. Spectral relative standard deviation: a practical benchmark in metabolomics. Analyst 2009;134(3):478–85.
[39] Pourakbar M, Farahani RZ, Asgari N. A joint economic lot-size model for an integrated supply network using genetic algorithm. Appl Math Comput 2007;189(1):583–96.
[40] Puentes K, Morales L, Pozo-Espin DF, Moya V. Enhancing control systems with neural network-based intelligent controllers. Emerg Sci J 2024;8(4):1243–61.
[41] Raissi M, Perdikaris P, Karniadakis GE. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. J Comput Phys 2019;378:686–707.
[42] Ramachandran P, Zoph B, Le QV. Searching for activation functions. [arXiv preprint] arXiv:1710.05941. 2017.
[43] Rizk-Allah RM, Hassanien AE. A hybrid Harris Hawks-Nelder-Mead optimization for practical nonlinear ordinary Differential equations. Evolutionary Intelligence 2022;15(1):141–65.
[44] Roul P, Biswal D. A new numerical approach for solving a class of singular two-point boundary value problems. Numerical Algorithms 2017;75:531–52.
[45] Sabir Z, Ali MR, Raja MAZ, Sadat R, Baleanu D. Dynamics of three-point boundary value problems with gudermannian neural networks. Evolutionary Intelligence 2023;16(2):697–709.
[46] Son H, Cho SW, Hwang HJ. Enhanced physics-informed neural networks with augmented lagrangian relaxation method (Al-pinns). Neurocomputing 2023;548:126424.
[47] Tato A, Nkambou R. Improving Adam optimizer. 2018.
[48] Tomasiello S, Macías-Díaz JE, Khastan A, Alijani Z. New sinusoidal basis functions and a neural network approach to solve nonlinear volterra–fredholm integral equations. Neural Comput Appl 2019;31:4865–78.
[49] Ul Rahman J, Nadeem U, Haider G, Rahbi al Y. Deep neural network-driven analysis of free vibrations in tapered beams. Appl Mech 2025;6(3):59.
[50] Wang S, Teng Y, Perdikaris P. Understanding and mitigating gradient pathologies in physics-informed neural networks. SIAM J Sci Comput 2021;43(5):A3055–A3081.
[51] Zhang Z, Wang Q, Zhang Y, Shen T, Zhang W. Physics-informed neural networks with hybrid Kolmogorov-Arnold network and augmented lagrangian function for solving partial differential equations. Sci Rep 2025;15:10523.

Author biography



Uzma Nadeem is an M.Phil. Mathematics student at the Abdus Salam School of Mathematical Sciences, GCU Lahore. Her research focuses on applying machine learning techniques to solve dynamical systems, including linear and nonlinear ODEs and PDEs. She holds a B.Sc. in Mathematics from the University of Education, Lahore. Her coursework includes PDEs, Functional Analysis, Linear Algebra, and ML fundamentals. Her current research is under review in peer-reviewed journals and she is expected to complete her degree by August 2025.



Malik Ahsin Iqbal is an M.Phil. Mathematics student at the Abdus Salam School of Mathematical Sciences (ASSMS), GCU Lahore, enrolled under a fully funded scholarship program since September 2023. He is expected to complete his degree by August 2025. He holds a B.Sc. in Mathematics from Abbottabad University of Science and Technology. His research focuses on Scientific Machine Learning, particularly the application of Physics-Informed Neural Networks (PINNs) for solving ODEs and PDEs. He also works on molecular property prediction and representation learning using Graph Neural Networks (GNNs) and their variants. His work has been submitted to three peer-reviewed journals and is currently under review.



Sadique Rehman is a Ph.D. candidate in Computational Mathematics at Kanazawa University, Japan, where he also completed his Master's degree in 2024. He holds a BS in Mathematics from Islamia College University, Peshawar, Pakistan (2020) and is currently supported by the MEXT scholarship. His research interests include computational fluid dynamics, soliton and wave theory, and the application of machine learning in engineering systems. His work has been published in peer-reviewed journals and presented at international conferences.



Aamir Farooq is a postdoctoral researcher at Zhejiang Normal University, China, specializing in Scientific Machine Learning, Physics-Informed Neural Networks (PINNs), and inverse problems. He holds a Ph.D. in Mathematics from Chongqing University, where he focused on matrix perturbation theory, and previously completed his MS from COMSATS University Islamabad. His research spans matrix perturbation theory, computational fluid dynamics, statistical signal processing, and AI-driven modelling of differential equations. He has published over 50 peer-reviewed articles in top SCI journals and previously worked as a postdoc at the Technical University of Liberec, Czech Republic. His current work involves integrating nonlocal differential equations with machine learning models under uncertainty, aiming to develop robust, interpretable tools for realworld scientific and engineering problems.



Hijaz Ahmad is an Associate Professor at Near East University, Nicosia, Cyprus. He earned his Ph.D. in Mathematics from the University of Engineering and Technology (UET), Peshawar, and his Master's degree from COMSATS University Islamabad. His research focuses on nonlinear systems, mathematical analysis, numerical methods, and partial and fractional differential equations (PDEs). He is actively engaged in developing innovative computational algorithms for solving complex PDEs arising in applied science and engineering. Dr. Ahmad has been recognized among the World's Top 2% Scientists for the years 2022, 2023, 2024 and 2025, as ranked by Stanford University and Elsevier.



Fuad A. Awwad is currently an Associate Professor in the Department of Quantitative Methods at King Saud University, Riyadh, where he formerly served as department head. He holds a Ph.D. in Applied Statistics from Lancaster University, UK, and has extensive teaching and consulting experience across academic, governmental, and healthcare sectors. His research interests span statistical modelling, epidemiology, case-control studies, and market research analytics. Recently, his focus has extended to AI-driven data analysis, leveraging statistical learning, big data analytics, and predictive modelling for real-world decision-making in public health and administrative sciences. He has contributed to multiple national statistical initiatives and published in international peer-reviewed journals.



Emad A.A. Ismail is an Associate Professor of Risk Management, Insurance, Mathematics, and Statistics at the College of Business Administration, King Saud University, Riyadh. He holds a Ph.D. in Insurance Studies from Cairo University and has extensive academic and professional experience in financial risk, insurance analytics, and actuarial modeling. His research interests span risk measurement, Bayesian inference, insurance fraud detection, and premium pricing models. Recently, his work has integrated AI-driven statistical methods, including ARIMA-wavelet forecasting, Bayesian analytics, and logistic regression models for fraud risk assessment and insurance decision-making. He has published in leading journals and presented at international conferences on health economics and insurance informatics.